

Track 1 : 멀티 에이전트 커맨드 센터

무엇에 대한 강의인가

Agentic 코딩 도구(CODEX & CLAUDE CODE) 활용에 있어 **단일 에이전트**를 넘어
검증 가능한 팀(멀티 에이전트)으로 **조율(orchestration)**하는 방법

간단한 이론 · 이후, 단계적 실습으로 이어집니다.

결론부터

멀티 에이전트 오케스트레이션이란 “AI 여러 개를 화려하게 켜는 것”이 아니라,
역할 분리 + 명확한 전달물 + 사람의 승인 & 검증 구조를 만드는 것이다.

deep

- 여러 에이전트에게 일을 맡기기 전에 **범위와 성공 기준**을 먼저 고정한다.
- 각 worker는 독립 작업 공간 & 독립된 작업 범위로 일하고, 결과는 **diff · commit · report**로 남긴다.
- main에 합치기 전, 사람이 **diff와 test log**를 보고 승인한다.

예: 특정 모듈 및 기능을 바꿀 때 출력 결과로 “완료했습니다”라는 말이 아니라, **변경 diff와 테스트 로그(exit code) & report**를 보고 승인한다.

강의 핵심

문제가 단순히 “에이전트가 코드를 못 짤다” 너머에 있음을 이해하고, 이를 구조적인 형태로 추적하기

에이전트는 이미 작은 코드 수정·문서 작성에 충분히 유용하다.

진짜 어려움은 작업 범위가 클 때, **단일 에이전트 & 멀티 에이전트가 작업한 일을 사람이 이해하고, 결정하기 위해 확인하는 비용**에 있다.

실제로 막히는 곳

- 어디를 **왜** 바꿨는지 추적이 어렵다
- 프로젝트 진행 시, 달성 목표가 누락된다
- 여러 AI가 같은 파일을 건드려 충돌이 난다
- 정밀한 테스트 없이 “문제 없음”이라고 말한다

그래서 필요한 것

- 작업 **범위**를 자른다
- worker별 **작업 공간**을 나눈다
- 결과를 **증거**로 남긴다
- 사람이 **승인**한다

포인트: “AI가 더 똑똑해질수록, 사람에게서는 더 강한 검증 구조가 필요하다.”

강의 목표

“통제 가능한 멀티에이전트 업무 구조”를 구축하고 경험한다.

프로젝트의 주도권을 가지면서 병렬 세션 에이전트 워크플로우가 동작하는 개념을 체험한다

흔한 오해	오늘의 방향
최신 모델을 많이 쓰면 생산성이 오른다	역할·격리·검증·승인 구조가 있어야 안전하게 오른다
AI가 알아서 끝까지 처리한다	AI는 제안·수정·검증 로그를 만들고, 사람이 승인한다
멀티 에이전트는 과한 작업 도구다	로컬 Git + worktree만으로 현실적인 최소 패턴을 만들 수 있다

오늘 배우는 것과 배우지 않는 것

배우는 것 (이론)

- Harness(작업장) 기초 개념
- Git Worktree와 병렬 세션
- 도구별 특징(Claude·Codex)
- 에이전트 작업 현황 추적 - HITL·Dashboard·운영 가이드

오늘 다루지 않는 것

- 외부 시스템 및 Tool 연동 자동화 개발
- 완전 자율 배포 / 장시간 무인 루프

오늘은 거대한 플랫폼을 이해하는게 아닌, 개발에 활용 가능한 “안전한 최소 패턴”을 경험하기

활용 개념

1. **Worktree**가 왜 필요한지 — worker별 격리 작업 공간을 이해한다.
2. **역할 분리** — 병렬 세션에서 planner·builder·tester·reviewer·approver의 TASK를 분리하는 개념을 이해하기
3. **증거** — `diff` · `test log` · `report` · `approval` 등 에이전트 구현 현황을 내가 이해할 수 있는 형태로 작성 체계를 구축하고 활용한다.
4. **도구 이해** — Agents 하네스 환경·모델·reasoning을 역할에 맞게 활용해야함을 이해하기

LLM에 대한 직관적인 이해

LLM은 “정답 계산기”가 아니라 “생성기”다

LLM이 잘하는 일	LLM이 약한 일
요구사항을 구조화한다	요구사항에 대해 누락없는 결과를 보장한다
코드 패턴을 읽고 초안을 만든다	오래된 맥락을 끝까지 정확히 유지한다
에러 로그를 요약한다	보안·권한 리스크를 완벽히 판단한다
테스트 후보를 만든다	큰 저장소 전체를 안전하게 일괄 수정한다

비유: LLM을 의심하는 습관이 필요하다. 개발 흐름은 빠르지만, 결과 품질은 검증 구조가 만든다.

Hallucination

“없는 말을 한다”보다 “제약사항 무시, 달성 누락, 테스트 없이 문제없음 턴 종료”

AI가 말하는 것	실제로 일어난 일
“테스트 통과했습니다”	스크립트 실행 누락 존재
“보안 문제 없습니다”	<code>.env</code> · token · 권한 범위 누락
“리팩토링 완료했습니다”	기존 API 호환성이 깨짐
“요구사항 반영했습니다”	핵심 케이스 1개만, edge case 누락

대응 원칙: 완료 선언에는 항상 명령 로그 · exit code · diff · report가 따라와야 한다. 말은 증거가 아니다.

에이전트를 코딩에 활용할 때, 중요 확인 사항

“에이전트한테 질문을 알맞게 던졌는가?”를 넘어서
“에이전트가 작업하는 환경이 현재 검증할 수 있는 구조로 잡혀있는가?”가 더 중요하다.

검증 가능한 증거 6종

- 어떤 명령을 실행했는가 → `logs/test.log`
- exit code는 무엇인가 → `0` 여부
- 어떤 파일이 바뀌었는가 → `git diff --stat`
- 요구사항과 diff가 맞는가 → 사람이 읽는다
- 실패 원인을 기록했는가 → `reports/*.json`
- 사람이 승인했는가 → `approval.json`

실무 예: “npm test 통과”라는 문장보다, `logs/test.log` 와 **exit code 0**이 신뢰의 근거다.

Agents

<https://www.kaggle.com/whitepaper-agents>

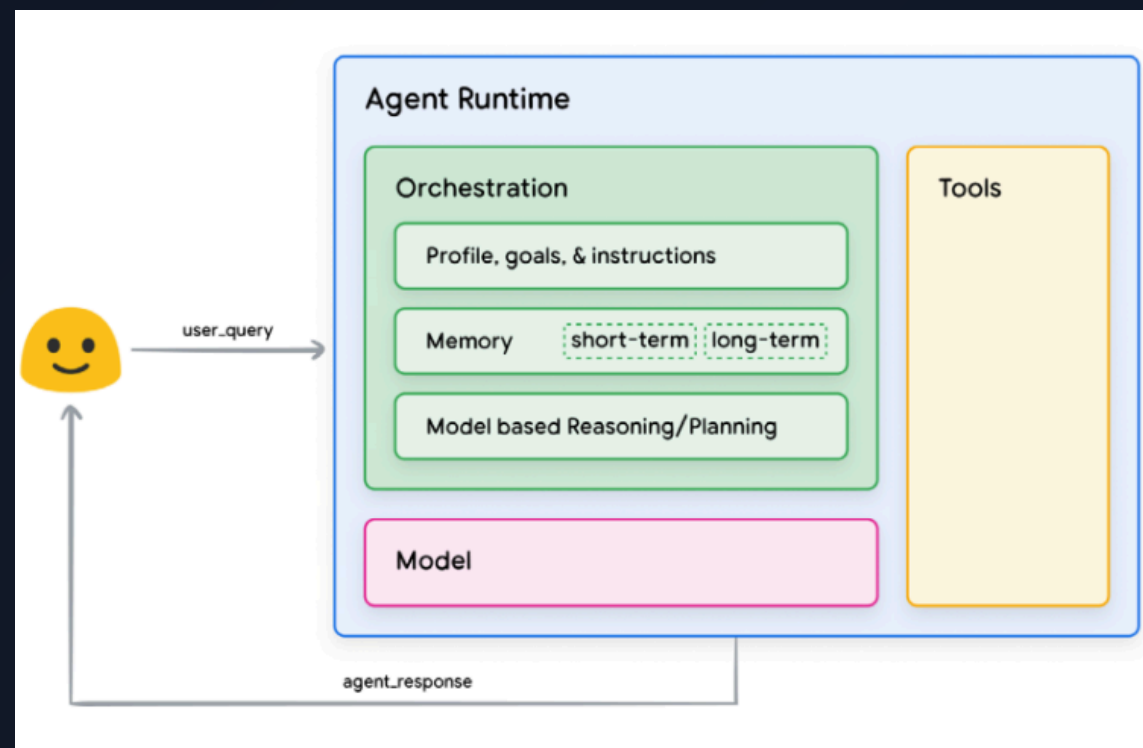
Google/Kaggle의 Agents 백서는 에이전트를 “모델 + 도구 + 오케스트레이션 레이어”로 설명한다.

모델(LLM) : 추론을 담당(LLM)

도구(Tool) : 외부 세계를 보고 행동하게 만든다.

오케스트레이션 : 관찰→추론→행동→피드백의 반복 루프 관리

즉 에이전트는 단순히 더 좋은 답변을 생성하는 LLM이 아니라, 목표를 향해 반복 실행되는 루프 시스템이다.



Agentic Coding이란

단순한 채팅(single inference)을 넘어, 하나의 에이전트가 “프로젝트 폴더 안에서 반복 루프”

Read files → Plan → Edit files → Run commands → Observe result → Fix

Claude Code 공식 문서는 이를 “코드베이스를 읽고, 파일을 편집하고, 명령을 실행하며 개발 도구와 통합되는 *agentic coding tool*”로 설명한다.

중요: CLI 에이전트는 실제 파일을 수정한다. 그래서 채팅형 AI보다 더 강한 하네스가 필요하다.

Harness Engineering - Agents 내 운영체제 구축

Harness = AI 작업장 (메모리, 권한설정, context 관리 등)

좋은 프롬프트 하나보다, AI(에이전트)를 감싸는 **메모리 · 규칙 · 검증 · 기록** 구조가 결과를 좌우한다.

검증과 격리의 관점에서의 예시

제어 환경 구성	역할	사람이 보는 증거
작업 지시 (<code>task.md</code>)	범위 · 성공 기준 고정	작업 지시서
실행 로그 (<code>logs/</code>)	실제 명령 결과 기록	검사 기록
변경 증거 (<code>diff</code>)	바뀐 코드 확인	작업 내역
구조화 보고 (<code>report</code>)	worker 결과 수집	보고서
사람 승인 (<code>approval</code>)	최종 merge 통제	승인표

비유: 하네스는 간단하게 말해서 **프롬프트 · 작업지시서 · 메모리 · 검사표 · 작업로그 · 승인표** 등의 묶음이다. 공장 · 연구소의 제품 품질 관리에 영향을 끼치는 모든 운영 환경이다.

하네스가 없을 때, “나쁜 요청”

왜 AI가 실패하는지 직관적으로 보이는 예

“우리 정산 도메인 쪽에 가끔 동시성 이슈(Race Condition) 터지는 거 Redis 분산락 걸어서 고도화해줘. 테스트 코드 작성해서 돌려보고, 커밋 메시지 알맞게 써서 때려줘.”

무엇이 빠졌나

- **범위가 없다** — `src/payment/` 인터셉터나 Global 미들웨어까지 건드려 사이드 이펙트를 만든다.
- **성공 기준이 없다** — 성능 저하(Lock Timeout) 대책이나 데드락 발생 시 Fallback 정책을 모른다.
- **수정 금지 파일이 없다** — 멀쩡한 `pnpm-lock.yaml` 을 초기화하거나 인프라 파이프라인 설정을 오염시킨다.
- **검증 명령이 조잡하다** — 도커 컴포즈로 Redis를 띄우고 테스트하지 않고 `pnpm test` 만 돌리고 끝낸다.
- **중단 기준이 없다** — Context Window가 가득 차고 API 비용이 터질 때까지 무한 디버깅 루프를 돌며 코드를 난도질한다.

하네스가 있다면, “좋은 요청”

같은 일도, 경계가 분명하면 안전해진다

좋은 작업 지시의 7요소

- 목표(Goal) · 범위(Scope)
- 수정 허용(Allow) / 금지(Deny) 파일
- 검증(Validate) 명령 · 산출물(Artifacts)
- 중단 조건(Fallback Barrier)

목표: SettlementService.calculate에 Redisson
기반 분산락(LeaseTime 3s) 적용 및 동시성 제어
작업 범위: src/settlement/domain/SettlementService.ts
검증: docker compose up -d redis &&
pnpm test src/settlement/tests/concurrency.spec.ts
산출물: git diff + docs/architecture/redis-lock.md
접근 금지: src/common/exception/, package-lock.json
중단: 1. 테스트 실패가 연속 3회 이상 반복 시
2. Lock획득 실패 로그가 해결되지 않고 롤백될 시
작업 즉시 중단 후 터미널 에러 로그를 dump하여 복귀

위 블록은 “현업 레포지토리 장애 방어 사례”입니다. AI 에이전트가 프로덕션 코드를 망가뜨리지 않도록 격리하는 경계를 보여주는 스냅샷입니다.

검증 신호가 있어야 자율성이 의미를 갖는다

AI가 스스로 고치려면 “맞고 틀림”을 읽을 수 있어야 한다

Claude 공식 문서: “*Give Claude a check it can run: tests, a build, a screenshot to compare.*” → 그래야 루프가 스스로 닫힌다.

검증 신호	쓰는 상황
<code>npm test</code> / <code>node scripts/check.js</code>	기능·단위 검증
<code>npm run lint</code> / <code>typecheck</code>	스타일·타입 안정성
screenshot 비교 / fixture diff	UI·출력 형식 보존

“테스트를 통과하게 해줘”라고만 하면 AI가 테스트를 약화·삭제할 수 있다. → “root cause를 고치고, 테스트를 숨기지 말라”고 지시한다.

Subagent / Hook / Skill 한 번에 이해하기

화려한 자동화 장난감이 아니라, “에이전트 워크플로우를 구성하는 부품이자 운영 안정화 장치”

기능	한 줄 정의	비유
Subagent	역할이 분리된 AI 작업자	전문 담당자
Hook	특정 시점에 자동 실행되는 명령	자동 검문소
Skill	반복 업무를 담은 매뉴얼	표준 작업 절차서

공통 목적: AI 작업을 **안전하고** · **반복 가능하게** 만드는 것. “무엇이고 왜 필요한가”까지만 이해합니다.

Subagent는 “역할 분리” + ,,,

별도 context와 권한을 가진 전문 작업자

Claude 문서: “Each subagent runs in its own context window with a custom system prompt, specific tool access, and independent permissions.” — 그리고 “결과 요약만 돌려준다.”

쓰기 좋은 경우

- 조사·로그 요약 (`log-analyzer`)
- 관점 리뷰 (`security-reviewer`, `test-reviewer`)
- 문서 동기화 (`docs-updater`)

굳이 안 써도 될 때

- 한 파일만 고치면 되는 경우
- 요구사항이 아직 불명확한 경우
- 역할이 1개로 충분한 경우

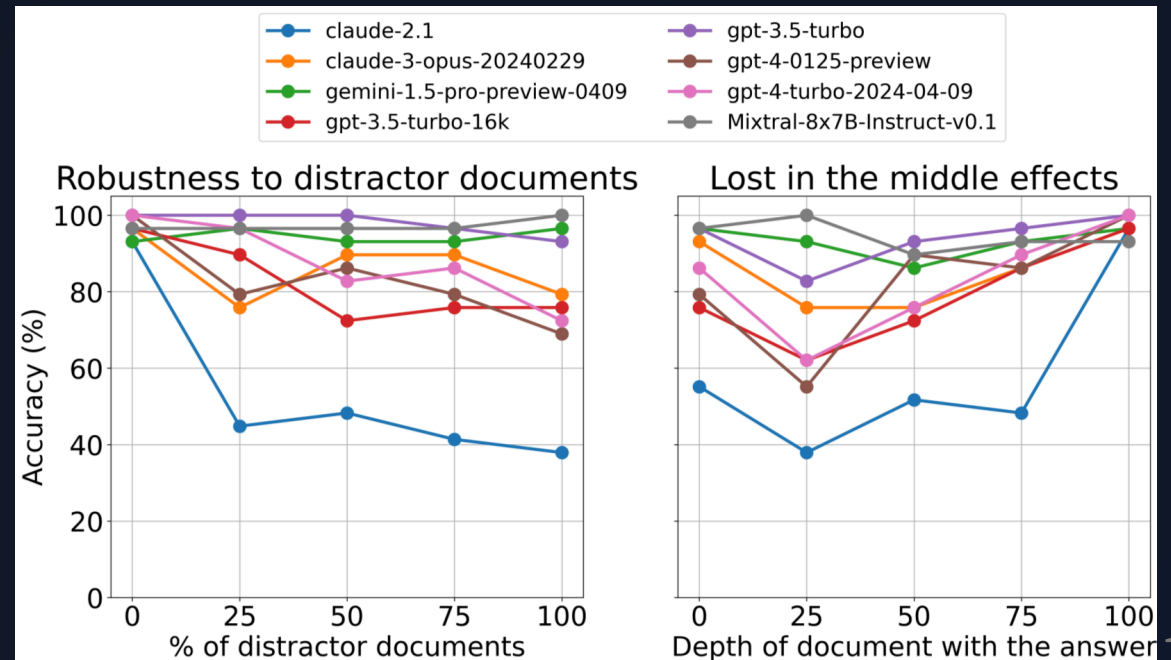
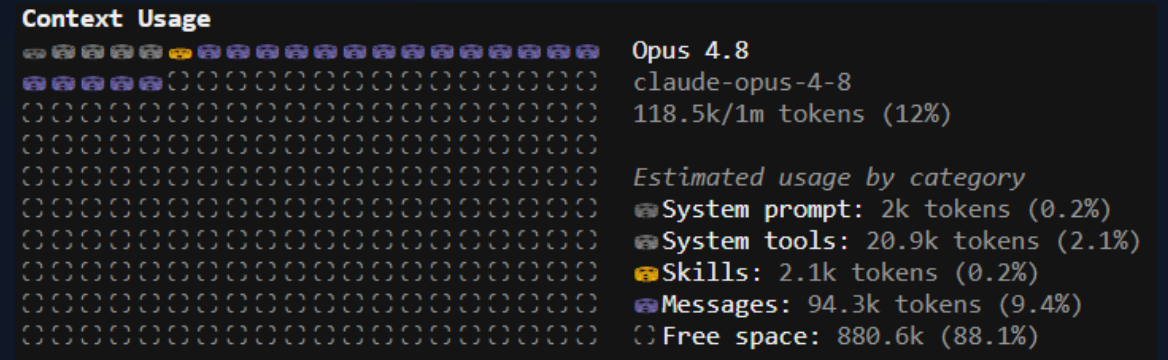
사례(문서 동기화): `docs-updater`가 diff를 보고 README 수정 후보를 만들고, reviewer가 코드·문서 불일치를 확인한다. → AI는 코드뿐 아니라 **변경의 흔적**을 문서까지 연결한다.

컨텍스트 윈도우(Context Window)

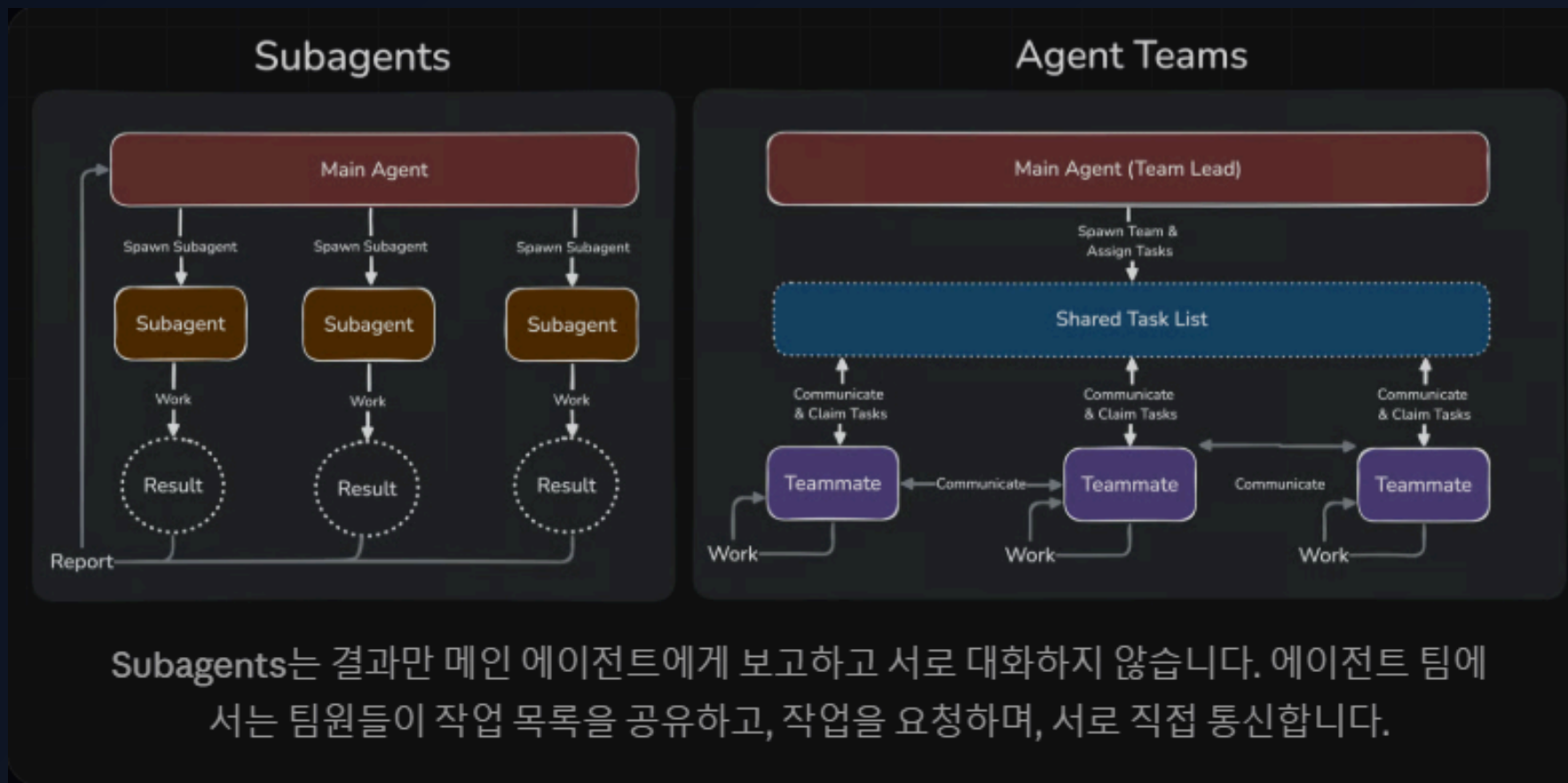
인공지능(AI)과 거대 언어 모델(LLM)이 한 번의 요청과 답변에서 기억하고 처리할 수 있는 정보의 최대 범위

텍스트를 쪼개는 단위인 토큰(Token)으로 측정되며, AI 모델의 성능을 결정하는 핵심 요소 중 하나

- 입력과 출력의 합: 컨텍스트 윈도우는 사용자가 입력한 프롬프트, 이전 대화 기록, AI가 생성한 답변, 시스템 지시사항을 모두 포함
- 한계 초과 시: 윈도우가 가득 차면, AI는 가장 오래된 대화나 정보부터 잊어버림
- 'Lost in the Middle' 현상: 컨텍스트 윈도우가 아무리 커도 문서의 중간에 있는 내용은 AI가 양 끝단보다 덜 정확하게 처리하는 경향



subagents vs agent teams



Hook은 “자동 검문소”다

사람이 잊어먹거나, LLM이 누락해도 정해진 점검이 항상 실행된다

Claude 문서: “*Hooks are user-defined shell commands that execute at specific points in Claude Code's lifecycle.*” — LLM의 선택이 아니라 **결정적으로** 실행된다.

Hook 예시	실무 목적
파일 수정 후 <code>lint</code> 실행	빠른 품질 확인
<code>.env</code> 변경 감지·차단	secret 노출 방지
위험 파일 수정 시 승인 요구	운영 위험 차단

Hook은 **shell command**를 실행한다. 그래서 “편한 자동화”가 아니라 “**권한 있는 검문소**”로 다뤄야 한다.
공유·운영 환경 배포 전 반드시 검토.

Skill은 “반복 업무 매뉴얼”이다

매번 복붙하는 프롬프트를 팀 표준 절차로

Claude 문서: “Create a `SKILL.md` file with instructions, and Claude adds it to its toolkit.” — 필요할 때 불러 쓰는 **재사용 워크플로 패키지**다.

Skill 예시	담는 절차
PR 리뷰	diff 읽기 → 위험 체크 → 리뷰 코멘트
장애 로그 분석	실패 로그 요약 → 원인 후보 → 수정 계획
릴리즈 노트	commit 요약 → 사용자 영향 → 배포 노트
보안 체크리스트	금지 파일·secret·권한 점검

핵심: 노하우를 사람 머릿속이 아니라 **파일로 표준화**해 팀 전체가 같은 품질로 반복한다.

Track 1

개념 이해만 — 에이전트 세션에서 에이전트가 목적에 맞게 동작하도록하는 부품

오늘 (이론)

- Subagent = 역할 분리라는 **감각**
- Hook = 자동 검문소이자 **위험 요소**라는 인식
- Skill = 반복 절차의 **표준화** 개념

오늘 다루지 않음

- 실제 Hook / Skill / Subagent 파일 작성
- 외부 도구 연동 자동화 구현
- 복잡한 자동화 체인

오늘의 핵심은 Worktree 기반 격리와 검증 가능한 산출물이다.

싱글 에이전트가 어려워지는 순간

한 세션에 모든 역할을 맡기면 생기는 일

무너지는 지점

- 컨텍스트 과부하 (맥락이 가득 차 성능 저하)
- 역할 충돌 (Builder가 구현한 코드에 대한 테스트 스크립트도 작성하면, 관점 분리 불가)
- 검증 흐름 · 비용 증가
- 반복 실패 누적

분리가 주는 것

- 역할별 독립 관점
- 실패 격리
- 검증의 객관성

실무 예: 한 세션에게 요구사항 정리·구현·테스트·리뷰를 **모두** 맡기면, AI는 자기 작업을 스스로 합리화한다. 예를 들어, 중복 데이터 저장 X를 위해 Builder가 id만 가지고 내부 구현을 진행했을때, Builder가 진행하는 테스트는 id만 가지고 진행될 수 있다. 별도의 tester를 두면, 구현을 확인하는 것이 아니라 요구사항이 실제로 지켜지는지 공격적으로 확인 가능(다른 id지만 같은 canonical URL, 같은 원문 등)

멀티에이전트란?

하나의 에이전트가 다 하지 않고, 역할로 나누는 구조

여러 에이전트가 “수다 떠는” 구조가 아니라, 각 역할의 **입력과 출력을 명확히** 하는 구조다.

역할	하는 일	산출물
planner	요구사항을 작업 단위로 분해	<code>task.md</code>
builder	정해진 범위만 구현	commit · diff
tester	명령 실행 · 실패 기록	<code>test.log</code>
reviewer	위험 · 누락 검토	<code>review.md</code>
human(approver)	최종 판단	<code>approval.json</code>

개발 업무는 원래 팀 단위 진행이다 — PM·개발·QA·리뷰어·승인자. AI 팀도 같은 분업을 따르면 덜 흔들린다.

병렬 세션을 통한 멀티 에이전트 활용?

Claude Code 엔지니어 리드, 보리스 처니

<https://x.com/bcherny/status/2017742743125299476?lang=en>

“3-5개의 Git worktree를 동시에 만들고, 각각 별도의 Claude 세션을 병렬로 실행하라.”

HashiCorp 공동창업자이자 Ghostty 개발자, 미첼 하시모토

<https://x.com/mitchellh/status/1963288264695321038>

“버그의 근본 원인을 분석할 때, 여러 GitHub 이슈에 걸쳐 동일한 프롬프트를 병렬로 실행한다.”

안드레 카파시

<https://x.com/karpathy/status/2026731645169185220?lang=en>

“여러 병렬 코드 인스턴스를 효율적으로 관리할 수 있는 적절한 도구, 메모리, 명령어를 갖춘 장기 실행 오케스트레이터를 구축하는 시대다.”

Django 공동 창시자, 사이먼 윌리슨

“parallel coding agent lifestyle” 언급

Amazon·Google 개발자, 스티브 예기

여러 터미널을 사람이 직접 관리하는 단계에서, 상위 에이전트가 20~30개 세션을 관리하는 단계로 넘어가기 위한 Gas Town이라는 오케스트레이터 개발

Vercel 창업자, 기예르모 라우흐

병렬 코딩 에이전트와 worktree를 전제로 한 CLI 에이전트 개발 환경 언급

Chrome Developer Experience 리드, 애디 오스마니

Claude Code와 Git worktree 병렬 운영 권장

PSPDFKit 창업자, 피터 스타인버거

5개의 Claude를 컨테이너에서 안전하게 병렬 실행하고, 결과를 IDE로 가져오는” 형태로 활용

병렬 세션이 필요한 이유

병렬의 핵심은 “속도”를 넘어 “격리”에 있다.

Claude 공식 문서가 밝히는 병렬 세션의 이유: “*speed up development, run isolated experiments, or start complex workflows.*”

병렬이 주는 것

- 작업 **범위** 격리
- **실패** 격리 (한 worker 실패가 전체로 안 번짐)
- **역할** 분리 (조사·구현·검증 context 분리)

병렬의 비용·위험

- 같은 파일 충돌
- 모델 비용 증가
- 책임 불명확 · 검증 누락

운영 원칙: 처음부터 4개 이상 돌리지 않는다. **1개 → 2개 → 3개** 순으로, “충돌을 줄일 만큼만” 늘린다.

Checkout과 Worktree

독립 Checkout

```
project-main/  
project-builder/  
project-tester/
```

- 같은 저장소를 여러 번 clone
- 각 폴더가 독립된 Git 저장소
- 격리는 강하지만 무겁고 관리가 번거로움

Git Worktree

```
project-main/  
project-builder/  
project-tester/
```

- 하나의 Git 저장소를 공유
- 작업 폴더·branch·HEAD는 분리
- 가볍고 생성·삭제가 빠름

여기서 여러 checkout이란 같은 폴더에서 `git checkout` 을 반복한다는 뜻이 아니라, 독립 작업 폴더를 여러 개 둔다는 뜻이다.

공통 구조 - 메인 세션

├ 구현 세션

├ 테스트 세션

└ 리뷰 세션

각 세션은 서로 다른 작업 폴더에서 실행

중요한 것은 clone인가 worktree인가가 아니라, 에이전트마다 독립된 작업 공간을 제공하는 것이다.

Git Worktree 활용

병렬 설계에 있어 가장 직관적이고 효율적이다

Git 공식 문서: “A git repository can support multiple working trees, ...”

Multiple Checkout	Git Worktree
저장소를 여러 번 clone	저장소 하나를 공유
디스크 사용량이 큼	상대적으로 가벼움
commit 공유에 fetch 필요	commit을 즉시 확인 가능
생성·정리가 번거로움	빠르게 생성·폐기 가능
강한 완전 격리에 적합	일반적인 병렬 개발에 적합

```
git worktree add ../builder -b feat/builder main
git worktree add ../tester -b test/builder main
```

Git Worktree를 활용한 실습 진행. 이유는 설정이 단순하고, worker 결과를 main에서 바로 검토할 수 있기 때문

Worktree를 쉽게 비유하면

같은 프로젝트의 “분리된 작업 책상”

작업 책상	누가 쓰나	무엇을 하나
<code>main</code>	기준 책상	검증·통합·승인의 기준
<code>builder</code>	구현 worker	정해진 범위만 수정
<code>tester</code>	검증 worker	명령 실행·실패 기록
<code>fixer</code> or <code>reviewer</code>	수정 or 리뷰 worker	실패 원인만 고침/밝힘

AI가 실패해도 **main** 책상은 더럽혀지지 않는다.

실패한 worker 책상에만 집중하거나, 비가역적이라면, 지우고 다시 만들면 된다.

결과물은 두 가지 통로로 전달한다

코드는 Git으로, 작업 과정은 메모리 파일(report.md 등)로 연결된다

예시

Builder

- 자신의 Worktree에서 코드 구현
- 변경 내용을 commit으로 고정
- 보고서·patch를 docs/building 등에 저장

Tester

- Builder branch 위에서 작업 시작
- Builder 코드를 검증하고 테스트 추가
- 테스트 결과·로그를 docs/test 등에 저장

Main / Human

- 보고서 + diff + 테스트 로그 확인
- 검증된 Tester branch를 main에 merge

코드 상태 전달

Branch

Builder 결과 위에서
Tester가 작업

변경 상태 고정

Commit

각 worker의 결과를
체크포인트로 기록

작업 과정 공유

docs md 파일 등
보고서·patch·로그
검증 증거 저장

결과는 “Git과 증거 전달”로

AI의 말이 아니라 branch · commit · diff · test evidence로 연결된다

변경 확인

```
git status
```

```
git diff
```

실제로 무엇이 바뀌었는지 확인

변경 고정

```
git commit
```

Builder·Tester의 결과를
되돌릴 수 있는 단위로 기록

최종 통합

```
git merge
```

검증된 Tester branch를
main에 반영

실습 2의 최종 통합

```
git merge --no-ff practice2/tester \  
-m "practice2: merge builder impl + tester validation"
```

`practice2/tester` 는 Builder branch 위에서 만들어졌으므로, Tester branch를 merge하면 **Builder 구현과 Tester 테스트가 함께 main으로 들어온다.**

확장 활용

실제 개발 환경으로 가도 근본 원리는 같다

간단 실습	실제
<code>report.md</code> 보고서	Pull Request 설명·리뷰 문서
<code>report.md</code> 테스트 로그	CI 테스트 결과
patch 파일	PR의 Git diff
Main Review 보고서	코드 리뷰·승인
사람이 실행하는 merge	보호된 main branch의 PR merge
Worktree별 AI worker	개발자·코딩 에이전트·CI worker

실습

`areport.md` + local branch + human merge

확장

Pull Request + CI + code review + protected branch

전체 운영 흐름

격리 → 구현 → 검증 → 선택적 통합

```
요구사항 분해
  ↓
worker별 Worktree 생성
  ↓
Claude Code / Codex 병렬 실행
  ↓
각 worker가 수정·테스트·commit
  ↓
main에서 diff와 테스트 검증
  ↓
merge
  ↓
실패한 Worktree는 폐기하고 다시 시작
```

운영 규칙

- 각 worker는 서로 다른 branch를 사용한다. 같은 파일을 동시에 수정하지 않도록 작업을 나눈다.
- worker는 main을 직접 수정하지 않는다. 최종 반영은 사람이 검토하고 승인한다.

병렬 에이전트 개발의 핵심은 많이 실행하는 것이 아니다.
작업을 격리하고, 검증된 결과만 main으로 가져오는 것이다.

충돌과 실패는 “정상”이다

병렬 작업에서 관리해야 할 일상적인 사건일 뿐이다

자주 일어나는 일

- 같은 파일·같은 줄 동시 수정 → 충돌
- 테스트 실패
- 엉뚱한 파일 수정
- 요구사항 일부 누락

대응 순서

- diff 확인 → report 확인
- task를 더 작게 축소
- 망가진 worker는 폐기·재생성
- 마지막은 사람이 승인

“한 번에 잘 됨”은 절대 일어나지 않는다.

“망가져도 어디를 보면 되는지 안다”로 구축하여, 발생하는 문제를 빠르게 해결한다.

멀티에이전트 오케스트레이션

최신 연구들의 공통 결론

- 멀티에이전트의 핵심 가치는 “AI를 여러 개 붙이는 것”이 아니라, 하나의 복잡한 일을 분해·병렬화·전문화·교차검증하는 데 있다.
- 에이전트 수가 많다고 성능이 좋아지는 것이 아니다. 병렬화할 수 있는 작업에 적절한 역할과 통제 구조를 적용할 때만 효과가 크다.

<https://research.google/blog/towards-a-science-of-scaling-agent-systems-when-and-why-agent-systems-work/>

- Google Research의 180개 에이전트 구성 비교에서도 병렬화 가능한 작업에서는 큰 성능 향상이 나타났지만, 순차적 계획 작업에서는 멀티에이전트가 오히려 성능을 크게 떨어뜨렸다.
- OpenAI 역시 단일 에이전트의 능력을 먼저 최대한 활용하고, 복잡한 논리나 도구 선택 문제가 실제로 발생할 때 멀티에이전트로 확장할 것을 권고

멀티에이전트 장점

1. 여러 작업을 동시에 처리할 수 있다

- 한 사람이 모든 조사를 순서대로 하는 대신, 여러 사람이 각자 맡은 부분을 동시에 처리하는 것과 같다.

2. 역할을 전문화할 수 있다

- 모든 일을 한 에이전트에게 시키는 대신, 각 에이전트가 잘해야 할 일을 명확히 제한할 수 있다.

3. 컨텍스트 한계를 분산할 수 있다

- 에이전트마다 별도의 작업 공간과 컨텍스트를 사용하므로, 하나의 대화창에 모든 정보를 억지로 집어넣지 않아도 된다.

4. 서로 다른 관점으로 결과를 교차검증할 수 있다

- 한 에이전트가 만든 답을 다른 에이전트가 그대로 믿지 않고, 비판·검증·재실행할 수 있다.

5. 복잡한 시스템을 모듈 단위로 확장하고 통제할 수 있다

- 업무가 늘어날 때 전체 에이전트를 다시 설계하지 않고 필요한 역할을 추가하거나 교체할 수 있다.

멀티에이전트 단점

1. 비용이 크게 증가한다

- 에이전트가 늘어나면 모델 호출, 컨텍스트 전달, 도구 실행, 결과 통합이 모두 반복된다.

2. 순차적인 작업에서는 오히려 성능이 떨어질 수 있다

- 앞 단계의 결과를 정확히 이어받아야 하는 작업은 여러 에이전트로 나눌수록 사고의 흐름이 끊길 수 있다.

3. 여러 에이전트가 같은 실수를 반복하거나 잘못된 합의를 만들 수 있다

- 에이전트 수가 늘어난다고 독립적인 지성이 자동으로 늘어나는 것은 아니다.

4. 오류가 발생했을 때 원인을 찾기 어렵다

- 결과가 틀렸다는 것은 알 수 있어도, 어느 에이전트의 어느 판단부터 잘못됐는지를 찾는데 많은 시간이 소요된다.

5. 운영 시스템이 매우 복잡해진다

- 멀티에이전트는 프롬프트 몇 개를 추가하는 문제가 아니라, 분산 시스템을 운영하는 문제에 가까워진다.

AGENTS 선택, 모델 선택, Reasoning 선택

이슈 흐름

날짜	사건	핵심 의미
4월 16일	Claude Opus 4.7 출시	공식 성능 향상과 사용자 체감 회귀 논란이 동시에 발생
4월 23일	GPT-5.5 출시	코딩·에이전틱 벤치마크에서 OpenAI가 크게 도약
5월 28일	Claude Opus 4.8 출시	순수 지능보다 정직성·검증·장기 작업 안정성을 강화
6월 9일	Claude Fable 5 공개	Mythos 계열의 강력한 능력을 제한적으로 공개 모델화
6월 12일	미국 정부의 Anthropic fable5 통제	출시 약 3일 만에 Fable 5·Mythos 5 전면 중단
6월 26일	GPT-5.6 제한 프리뷰	미국 정부 요청을 반영해 승인된 일부 기관에만 제공
6월 30일	Anthropic fable 규제 해제·Sonnet 5 출시	Fable 복귀 준비와 보급형 에이전트 모델 확대 & Fable 5 구독 프로모션 종료 선언
7월 7일	Fable 5 구독 프로모션 종료일	정액 구독 포함 모델에서 별도 사용량 과금 모델로 전환할건데 7월 12일까지 늘려주겠다!
7월 9일	GPT-5.6 일반 공개	Sol·Terra·Luna 모델군 출시
7월 24일	Claude Opus 5 출시	Fable 5에 근접한 성능을 절반 수준 가격(\$5/\$25)에 제공
7월 30일	GPT-5.6 Terra·Luna 대폭 가격 인하	OpenAI가 출시 3주 만에 Luna 가격을 80%, Terra 가격을 20% 인하
현재	가격 경쟁 심화	고성능 frontier → 실무 주력 → 대량·저비용 계층화

1. Opus 4.7 회귀와 GPT-5.5의 도약

Opus 4.7

공식 평가: 장기 작업·소프트웨어 엔지니어링 강화

사용자 체감: 토큰 소비·속도·지시 이행 관련 능력이 4.6보다 뛰보

GPT-5.5

공식 평가: 에이전트 코딩·터미널 작업 도약

사용자 체감: Opus 4.7보다 빠르고 효율적이며 강력

2. Mythos에서 Fable 5로, 그리고 3일 만의 중단

최고 성능 모델이 곧바로 일반 공개될 수 없는 시대

Mythos Preview

- 강력한 취약점 탐지 능력
- 제한된 기관만 사용

Fable 5

- Mythos 계열 능력의 공개형·안전조정 버전
- 6월 9일 공개
- 6월 12일 미 정부 명령으로 전면 중단

3. Opus 4.8과 Sonnet 5의 등장

“압도적인 지능 향상”보다 신뢰성과 가격 효율을 노린 모델

Opus 4.8

= 고난도 작업·정직성·자기검증·대규모 오케스트레이션

Sonnet 5

= Opus에 가까운 능력을 더 낮은 비용과 높은 속도로 제공

사용자 체감

→ Fable 5 을 맛본 후라서...

실제 제품 포지션

→ Opus 4.8은 신뢰성 중심

→ Sonnet 5는 비용 대비 성능 중심

결과

→ 벤치마크는 향상됐지만 “완전히 다른 수준”이라는 체감은 약하다

4. GPT-5.6 공개 전 미국 정부의 개입

Fable처럼 완전히 금지된 것은 아니었다

OpenAI는 6월 26일 GPT-5.6을 발표했지만, 미국 정부의 요청을 반영해 일부 승인된 파트너와 기관에만 제한적으로 제공했다. OpenAI는 이와 같은 정부 사전 접근 절차가 장기적인 기본 정책이 되어서는 안 된다는 입장도 밝혔다.

GPT-5.6 발표

- 즉시 전 세계 공개 X
- 승인된 일부 조직에 제한 프리뷰
- 추가 안전성 평가
- 7월 9일 일반 공개

미국 정부의 국가안보 우려로 공개 일정과 접근 범위가 제한됐으며, OpenAI는 제한 프리뷰를 거친 뒤 일반 공개했다.

백악관은 이후 민간 기업의 모델 출시에 정부의 공식 승인이나 ‘그린라이트’가 필요한 것은 아니라고 해명했다. 즉 법적 사전허가제라기보다 정부 압력과 기업의 협조로 만들어진 **사실상의 단계적 배포**에 가까웠다.

5. Fable 5 복귀와 구독제 제외

모델이 사라진 것이 아니라 과금 방식이 바뀌었다

미국 상무부는 6월 30일 Fable 5와 Mythos 5에 대한 제한을 해제했고, Anthropic은 추가 안전 필터를 적용한 뒤 7월 초 Fable 5 접근을 복구했다. 위험 가능성이 높은 요청은 Opus 4.8로 우회시키는 방식도 적용(세이프티 가드)됐다.

이후 Fable 5는 Pro·Max·Team 등의 구독 한도 안에서 무료로 사용할 수 있는 프로모션 형태로 제공됐지만, 이 혜택은 7월 12일 오후 11시 59분(미국 태평양 시간)에 종료되도록 정해졌다. 이후에는 정액 구독에 포함되지 않고 별도 사용량 크레딧으로 이용하며, 공개된 가격은 입력 100만 토큰당 10달러, 출력 100만 토큰당 50달러다.

7월 12일 이전

Claude 구독 한도 안에서 Fable 5 사용

7월 12일 이후

일반 구독 포함 X

→ 별도 크레딧·사용량 기반 과금

Fable 5의 구독 포함 프로모션이 종료되고, 고가의 별도 사용량 과금 모델로 전환됐다.

6. GPT-5.6의 등장(7월9일)

하나의 모델이 아니라 세 가지 가격·성능 계층

모델	역할	적합한 작업
Sol	최상위 플래그십	고난도 코딩·과학·사이버 보안·장기 에이전트 작업
Terra	성능·비용 균형	일반 개발·기업 업무·대량 지식 작업
Luna	빠르고 저렴한 모델	반복 요청·간단한 코드·대량 처리

Sol에는 더 깊게 추론하는 `max` 와 여러 서브에이전트를 동원하는 `ultra` 모드가 추가됐다. Terra는 Sol 가격의 약 절반, Luna는 Terra보다도 저렴한 계층으로 소개됐다.

초기 반응

강점

- 빠른 에이전트 코딩 - Sam Altman은 에이전트 코딩에서 토큰 효율이 54% 개선됐다고 주장했지만 비교 기준은 명확히 공개하지 않았다.
- 작업당 비용은 Fable 5의 약 3분의 1 수준 (Artificial Analysis)
- max·ultra를 통한 추론량 선택

주의점

- 공개 직후라 독립적인 장기 평가가 부족
- ultra의 다중 에이전트 사용은 비용·통제 부담 증가
- 절대 지능은 Fable 5가 근소하게 앞설 가능성이 있지만, Sol은 성능·속도·비용 균형에서 강하다

어떤 도구를? 어떤 모델을?

도구 비교의 올바른 관점

내가 가진 문제를 무엇이 가장 적절하게 해결할 수 있는가-에 집중해야한다.

관점 1

“Claude가 더 좋아? Codex가 더 좋아? fable? sonnet? opus? sol?”

관점 2

“지금 이 문제 or 프로젝트는 어떤 모델/도구를 활용해야할까?”

도구	강점
Claude Code	lead · planner
Codex CLI	builder · fixer · reviewer 자동화
Gemini / Antigravity	researcher · log scan · 동향·보안 비교

벤치마크

<https://www.tbench.ai/leaderboard>

터미널 환경에서 실제 개발/운영 작업을 얼마나 잘 수행하는지 평가한 벤치마크 리더보드

- 코딩 품질을 포함하여 터미널 샌드박스에서 모델/에이전트가 컴파일, 디버깅, 시스템 관리, 파일 조작, 보안 취약점 해결, 데이터 처리 같은 다단계 작업을 수행하는지를 다각화하여 평가

Claude Code의 역할

레포 안에 상주하는 Lead Engineer

Claude Code는 단순한 코드 생성기가 아니라,
레포 구조를 이해하고 작업을 쪼개며 검증 흐름까지 설계하는 **agentic coding tool**이다.

Claude Code가 잘하는 일

맥락 이해 · 설계 · 운영 흐름

- 요구사항 분해와 작업 순서 설계
- 처음 보는 대형 코드베이스 탐색
- 멀티파일 리팩터링 방향 제안
- 테스트 전략 · 리뷰 기준 · 검증 루프 설계

Claude Code의 주의점

강력한 자동화에 비례하여 범위를 먼저 잡아야 한다

위험	운영 원칙
너무 큰 범위를 한 번에 수정	먼저 plan mode로 영향 범위와 파일 목록을 받는다
context 오염	긴 작업은 단계별로 끊고 새 세션을 활용한다
과도한 자율 실행	default/manual 또는 plan에서 시작한다
리팩터링 남발	non-goal과 금지 범위를 먼저 적는다

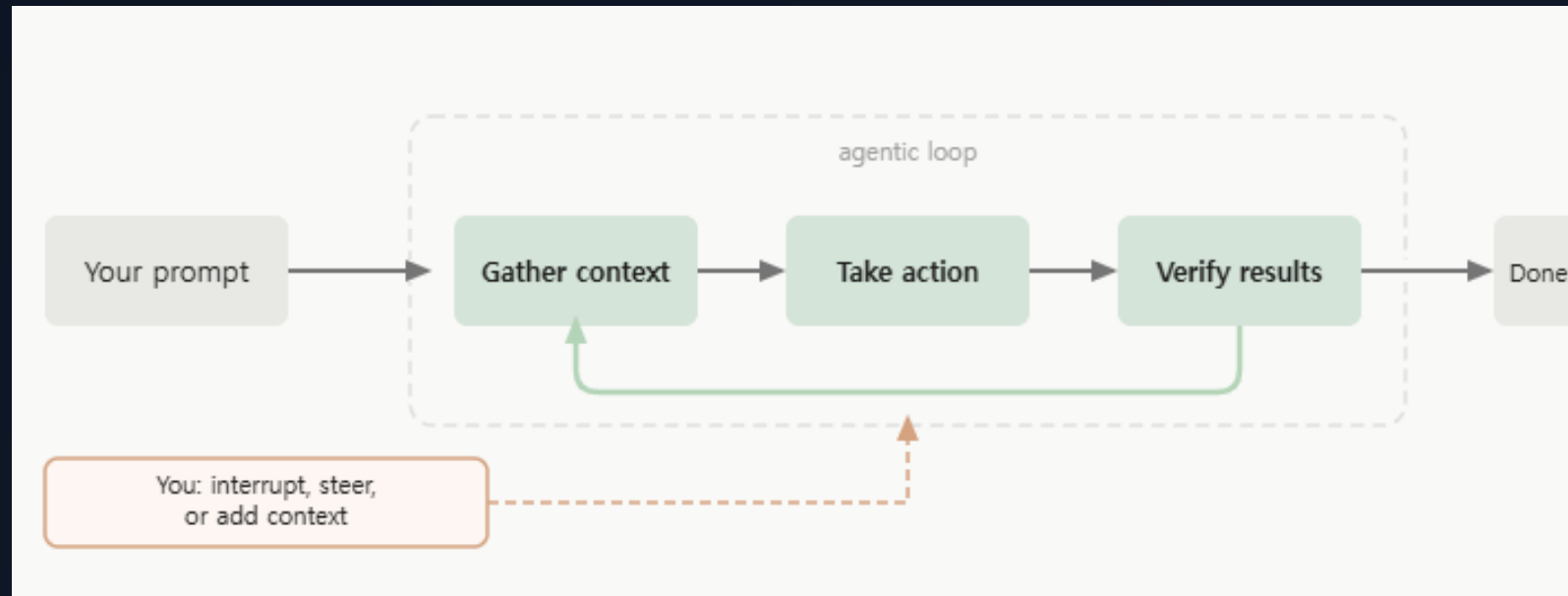
Claude Code 활용

“코드를 바로 고쳐”가 아니라 “먼저 구조를 파악해”

claude code docs에서 권장하는 좋은 사용 흐름은 다음과 같다.

Explore → Plan → Scope Lock(권한) → Implement → Test(검증) → Review

Claude Code에게 바로 구현을 맡기기보다,
먼저 **관련 파일, 영향 범위, 수정 계획, 검증 계획**을 요구하는 게 좋다.



Claude Code workflow 운영 모드

권한은 “속도”가 아니라 “실패 비용” 기준

- **default/manual**: 행동마다 사람 확인
- **acceptEdits**: 파일 편집은 허용, diff로 사후 검토
- **plan**: 수정 없이 탐색과 계획만 수행
- **auto**: 반복 승인 부담을 줄임
- **dontAsk**: 허용된 작업만 수행, 나머지는 거부
- **bypassPermissions**: 격리된 컨테이너·VM에서만 사용

권한을 많이 줄수록 똑똑해지는 게 아니라, 실패 반경이 커진다.

Codex의 역할

Diff · Review · CI · PR에 강한 Worker

Codex는 레포 전체의 운영 방식을 설계하는 도구라기보다,
정해진 작업을 작은 변경 단위로 수행하고 diff로 검토하기 좋은 도구다.

Claude Code가 “어떻게 쪼개서 갈까?”에 강하다면,
Codex는 “이 실패 테스트를 고쳐라”에 강하다.

Codex가 잘하는 일

작은 작업을 패치로 만들어오는 도구

- 실패 테스트 원인 분석
- 작은 버그픽스
- PR diff 리뷰
- PR에서 리뷰·수정 흐름 자동화

Codex의 주의점

넓은 목표보다 좁은 작업에 강하다

상황	위험	대응
작업 범위가 넓다	엉뚱한 파일 수정	수정 가능 파일과 금지 파일 명시
테스트 기준이 없다	그렇듯하지만 깨지는 코드	검증 명령을 먼저 지정
리뷰 기준이 없다	사소한 코멘트 남발	P0/P1 기준으로 제한
로그만 던진다	원인 추정이 흔들림	로그 + 재현 명령 + 기대 동작 제공

Codex 활용

범위를 좁힐수록 품질이 올라간다

나쁜 지시:

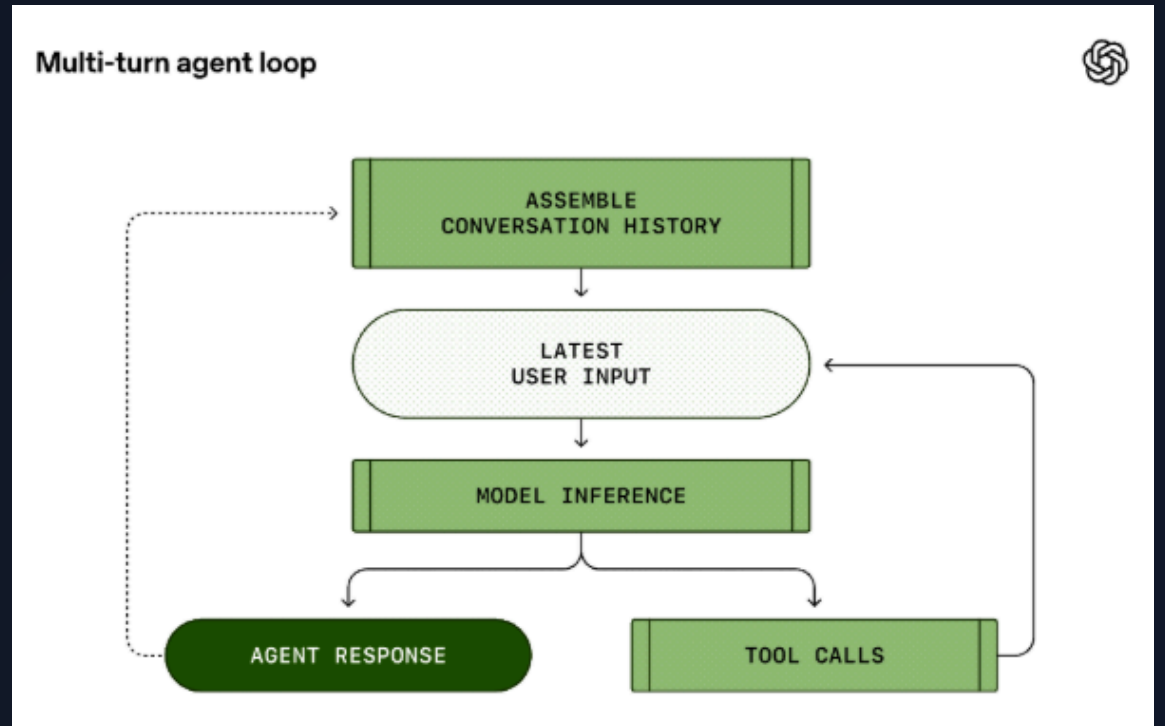
이 프로젝트 전반적으로 개선해줘.

좋은 지시:

`tests/test_auth.py` 실패만 고쳐라.

수정 가능 파일은 `src/auth/*` 와 해당 테스트 파일로 제한한다.

완료 후 `pytest tests/test_auth.py` 로 검증해라.



Antigravity의 역할

IDE · Terminal · Browser를 오가는 관제형 에이전트

Antigravity는 단순한 CLI 도구라기보다,
에이전트가 코드를 만들고 브라우저에서 확인하는 과정을 관제하는 플랫폼에 가깝다.

특히 UI, 풀스택, 프론트엔드 실습에서 직관적이다.

Antigravity 활용

“코드를 봤다”보다 “동작을 봤다”

Claude Code는 레포 안의 테크리드에 가깝고,
Codex는 diff를 만드는 구현 worker에 가깝다.

Antigravity는 에이전트들이 **editor, terminal, browser**를 오가며 일하는 과정을 보는 관제탑에 가깝다.

세 도구의 핵심 차이

내부 동작보다 “검토 표면”이 중요하다

관찰 프레임	대표 도구	사람이 검토하는 것	적합 작업
맥락·계획 중심	Claude Code	계획, 영향 범위, 파일 구조	대형 레포 탐색, 아키텍처, 리팩터링
diff·PR 중심	Codex	patch, diff, PR comment, CI 결과	버그픽스, 테스트 실패 수정, PR 리뷰
artifact·브라우저 중심	Antigravity	screenshot, walkthrough, browser recording	UI 구현, 풀스택 프로토타입, 시각적 검증

최종 판단 기준

모델 성능 순위가 아니라 작업 성격으로 고른다

질문은 하나다.

지금 작업은 계획이 중요한가,
코드 diff 수정, 리뷰, 최적화가 중요한가,
실제 동작 검증이 중요한가?

계획과 구조가 중요하면 **Claude Code**.

패치와 리뷰가 중요하면 **Codex**.

브라우저에서 보이는 결과가 중요하면 **Antigravity**.

모델 선택 기준

도구보다 먼저 “모델 역할”을 나눠야 한다

같은 코딩 에이전트라도 모델 등급에 따라 맡겨야 할 일이 다르다.

역할	필요한 능력
Lead / Architect	깊은 추론, 구조 파악, 장기 계획
Builder / Fixer	구현, 디버깅, 테스트 수정
Reviewer / Judge	diff 검토, 위험 탐지, 반례 찾기
Subagent / Batch Worker	빠른 반복, 로그 요약, 단순 수정

Claude 계열 모델 역할

Claude Code 안에서의 실무적 포지션

모델	수준	가장 적합한 역할
Claude Fable 5	최상위	장시간 autonomous agent, 고난도 설계, 복잡한 rescue 작업
Claude Opus 5	상위	대형 레포 분석, 아키텍처, 복잡한 agentic coding
Claude Sonnet 5	기본 주력	일반 개발, 멀티파일 구현, 계획+코딩 균형
Claude Haiku 4.5	경량	로그 요약, 단순 수정, 빠른 반복, 보조 subagent

Claude 모델 선택 감각

깊이와 비용을 기준으로 나눈다

상황	추천 모델
논문/대형 프로젝트 rescue, 구조적 문제 해결	Fable 5
새 레포 분석, 설계, 리팩터링 방향 수립	Opus 5
대부분의 Claude Code 실무 작업	Sonnet 5
빠른 반복, 문서 요약, 테스트 로그 정리	Haiku 4.5

Claude는 “레포를 이해하고 운영 흐름을 잡는 모델”로 쓰는 게 좋다.
가장 강한 모델은 가장 어려운 판단 구간에만 쓴다.

GPT 계열 모델 역할

Codex · ChatGPT · API에서의 실무적 포지션

모델	수준	가장 적합한 역할
GPT-5.6 Sol	최상위 코딩 주력	복잡한 기능 구현, 대규모 리팩터링, 아키텍처 설계, 장기 에이전트 작업
GPT-5.6 Terra	균형형 주력	일반 기능 개발, 코드 리뷰, 테스트 작성, 문서화, 일상적인 Codex 작업
GPT-5.6 Luna	고속·저비용	subagent, 반복 수정, 단순 구현, 대량 코드 분석, 병렬 작업
GPT-5.5	범용 최상위	복잡한 코딩, 심층 리서치, 설계 검증, 장문 분석, 현실 문제 해결
GPT-5.4	일반 실무형	일상적인 코딩, 버그 수정, 코드 설명, 소규모 기능 개발
GPT-5.4 Mini	경량 실무형	단순 코딩, 로그 분석, 테스트 보조, 형식 변환, 저비용 반복 작업

GPT 모델 선택 감각

“검토 가능한 패치”와 “판정”에 강하다

작업 유형	권장 모델
대규모 구현·복잡한 리팩터링·멀티파일 수정	GPT-5.6 Sol
일반적인 기능 개발·리뷰·테스트·문서화	GPT-5.6 Terra
병렬 subagent·반복 작업·속도가 중요한 작업	GPT-5.6 Luna
코딩과 리서치·설계 판단을 함께 수행	GPT-5.5
일반적인 코딩과 버그 수정	GPT-5.4
단순 작업·대량 처리·비용 최적화	GPT-5.4 Mini

중요하고 복잡한/방대한 구현은 GPT-5.6 Sol, 일반 개발은 GPT-5.6 Terra, 병렬 작업과 반복 처리는 GPT-5.6 Luna가 적합?

GPT 계열은 “diff, judge, structured output, review”에 배치하기 좋다.
Codex에서는 GPT-5.3-Codex를 구현 worker로 보는 게 자연스럽다.

Reasoning Level

같은 모델에 얼마나 많은 추론 자원을 배정할 것인가

Level	추론 정도	속도	토큰 부담	적합한 작업
Low	짧고 가볍게 판단	매우 빠 름	●○○○○○	단순 수정, 검색, 반복 작업
Medium	속도와 정확도의 균형	빠름	●●○○○○	일반 코딩, 문서 작성, 일상 업 무
High	여러 단계와 예외까지 검토	보통	●●●○○○	복잡한 구현, 디버깅, 설계
Extra High	장시간 탐색·검증	느림	●●●●○○	보안 리뷰, 대형 코드 분석, 심 층 연구
Max	한 에이전트가 최대한 깊게 추 론	매우 느 림	●●●●●●	가장 어려운 문제, 설계 검증, 반례 탐색
Ultra(codex) & Ultracode(claude)	최대 추론(Max) + 작업 자동 분해·위임	작업별 상이	●●●●●● × Agents(●●●●○○)	병렬 조사, 다면 리뷰, 대규모 복합 작업

Medium이 기본값이다. 작업이 어려워질 때만 High → Extra High → Max로 높인다. Ultra는 단순한 추론 단계가 아니라 서버에이전트 워크플로다.

OpenAI는 낮은 effort가 속도와 토큰 절약에 유리하고, 높은 effort가 복잡한 작업의 품질을 높일 수 있다고 설명. 모델은 작업 난도에 따라 토큰을 적응적으로 사용하므로, 같은 설정이라도 항상 동일한 토큰을 소비하지 않음.

토큰 증가 대비 정확도 상승

추론량을 높인다고 정확도가 선형으로 상승하지는 않는다

변경	토큰·지연 증가	단순 작업의 품질 상승	복잡한 작업의 품질 상승	비용 대비 효율
Low → Medium	낮음~중간	작음	큼	매우 좋음
Medium → High	중간	거의 없음	중간~큼	좋음
High → Extra High	큼	거의 없음	중간	작업 의존
Extra High → Max	매우 큼	없거나 악화 가능	작음~중간	낮음
Max → Ultra	에이전트 수만큼 증가 가능	불필요	분해 가능한 작업에서 큼	구조 의존

정확도 곡선은 보통 초반에는 크게 상승하고, 뒤로 갈수록 상승 폭이 줄어드는 형태다. 따라서 가장 높은 설정을 고정하지 않고, 필요한 최소 수준을 선택한다.

OpenAI도 `medium` 을 품질·속도·비용의 균형점으로 제시하며, `xhigh` 와 `max` 는 자체 평가에서 명확한 이득이 확인되는 경우에만 사용하라고 권장한다. ([OpenAI 개발자][2])

추론 토큰의 효과는 문제 난도에 크게 좌우된다. Test-time compute 연구에서는 문제별로 계산량을 적절히 배분하면 단순한 다중 샘플링보다 4배 이상 효율적일 수 있었지만, 다른 연구에서는 불필요하게 긴 추론이 오히려 정확도를 떨어뜨리는 역스케일링도 확인됐다.

Reasoning 선택 기준

- Claude와 Codex는 모두 Low → Medium → High → Extra High(xhigh) → Max라는 유사한 effort 단계를 제공. 각 단계가 담당하는 역할은 비슷하지만, 내부 계산량과 절대 성능이 표준화된 것은 아니다.

“가장 높은 설정”이 아니라 “통과 가능한 가장 낮은 설정”

작업	권장 Level
문구 수정, 파일 탐색, 단순 변환	Low
일반 기능 구현, 테스트 작성	Medium
복잡한 버그, 여러 파일 수정	High
보안·아키텍처·대형 코드 리뷰	Extra High(Xhigh)
최종 설계 검증, 치명적 장애 분석	Max
조사·구현·테스트·리뷰를 병렬 분리	Ultra

1. Medium으로 시작
2. 결과가 부족하면 High
3. 어려운 문제만 Extra High / Max
4. 작업이 독립적으로 분리될 때만 Ultra

멀티에이전트 아키텍처

리더형 · 파이프라인형 = 필수 / 위원회형 · 경쟁형 = 확장

OpenAI Agents SDK는 오케스트레이션을 두 갈래로 정의한다.

방식	의미	Track 1 적합성
LLM 오케스트레이션	AI가 다음 행동을 동적으로 판단	유연하지만 예측 어려움
코드 오케스트레이션	코드가 순서를 고정	<i>“more deterministic and predictable”</i>

고정형(코드) 흐름을 기본으로 둔다. (왜 더 예측 가능한지의 세부는 강의용 부연이며, 공식 문서는 위 한 문장으로 결과만 단언한다.)

리더형 (마스터-워커)

중앙 리더가 일을 나누고, 결과를 모은다

Human → Lead/Main → Worker(s) → Report → Human Approval
(계획·분배) (제한 구현) (diff·log) (merge 결정)

장점

- 책임자가 명확
- 결과 통합이 쉬움

주의

- 리더 판단이 틀리면 전체가 흔들림

사례(기능 구현형): 대시보드 버튼이 항상 “실행”으로만 보인다 → planner가 완료 기준 작성 · builder가 작은 수정 · tester가 로컬 검증 · reviewer가 불필요한 변경 점검. **최소 패턴을 익히기 가장 좋은 형태.**

파이프라인형 (고정 흐름)

한 단계의 출력이 다음 단계의 입력이 된다

요구사항 → 계획 → 구현 → 검증 → 리뷰 → 승인

OpenAI Agents SDK: “Chaining multiple agents by transforming the output of one into the input of the next.”

장점

- 단계별 완료 기준이 명확
- 고정 순서라 재현·검증이 쉬움

주의

- 앞 단계가 부실하면 뒤가 흔들림

사례(버그 수정형): `npm test` 실패 → log triager가 로그 요약 → fixer가 root cause 수정 → tester가 재검증. 흐름은 실패 로그 → 원인 → 최소 수정 → 재검증으로 고정한다.

위원회형과 경쟁형은 “확장 개념”

모든 작업이 아니라, 중요한 변경·후보 비교에만 쓴다

위원회형

여러 관점이 동시에 리뷰.

사례(공통 인증 모듈): security · test · maintainability reviewer로 관점을 나눈다. 구현자 한 명의 자기평가로 끝내지 않는다.

경쟁형

여러 후보를 만들고 고른다.

사례(레거시 마이그레이션): 한 번에 전부 바꾸지 않고, 수정 후보를 만들어 영향·위험을 비교한 뒤 1~2개만 적용·확대한다.

비용·조정 부담이 크다 → 기본 실습 구조가 아니다. 중요한 변경에서만 선택적으로.

Diff 중심 사고

핵심

“수정했습니다”는 증거가 아니다 — **git diff**가 증거다.

“테스트했습니다”도 증거가 아니다 — **test log**와 **exit code**가 증거다.

```
git diff --stat # 어떤 파일이 얼마나 바뀌었는지 한눈에
```

실무 예: 공통 util만 바꾸기로 했는데 diff에 `package.json`·`lockfile`까지 들어와 있으면 **승인하지 않는다**. 범 위 밖 변경은 곧 리스크다.

HITL — 사람이 최종 승인하는 구조

AI는 제안하고 증거를 모은다 · 최종 책임은 사람

AI가 대신 내릴 수 없는 판단

- 고객 데이터 · 사내 보안 정책
- 배포 · 운영 영향
- 요구사항 충족 여부
- QA · 운영과의 합의

```
{  
  "run_id": "run_001",  
  "status": "approved",  
  "approver": "human",  
  "notes": "diff와 test.log 확인 완료"  
}
```

사례(리뷰 보강형): AI reviewer가 요구사항 · 테스트 누락 · 보안 · 유지보수성을 선검토하고, **사람은 최종 merge를 소유**한다. AI는 사람 리뷰를 대체하지 않고, 그 전 잡음과 누락을 줄인다.

Dashboard가 필요한 이유

터미널 로그는 길고 흩어진다 → 한곳에 모아 “승인”한다

대시보드가 모으는 것

- 요구사항 · worker 상태
- 변경된 파일 목록
- 테스트 결과(exit code)
- 위험 체크리스트
- 승인 상태

핵심

Dashboard는 예쁜 UI가 아니라 승인 관제판이다.

시각적 화려함보다 “검증 표시”가 우선. 사람이 5초 안에 승인/보류를 판단할 수 있어야 한다.

이론에서는 “무엇을 모으고 왜 필요한가”까지만 다룬다. 실제 화면 구현은 이론 시간의 범위가 아니다.

실패 루프를 끊는 기준

같은 실패를 계속 반복시키지 않는다

같은 테스트가 2번 실패하면:

- 1) 멈춘다
- 2) 실패 로그를 20줄 이내로 요약한다
- 3) 원인 후보를 2~3개로 나눈다
- 4) task를 더 작게 다시 쓴다
- 5) 새 context에서 fixer를 다시 시작한다

실패한 대화를 계속 끌고 가면 **context가 오염된다**. 자동화의 출발점은 “더 많은 권한”이 아니라 “더 명확한 완료 기준”이다.

이론 파트 최종 정리

핵심 7문장

1. AI의 말보다 `diff` 를 본다.
2. 큰 일은 작은 task로 쪼갬다.
3. worker마다 작업 폴더를 나눈다.
4. 테스트 없는 자동화는 자동화가 아니다.
5. 실패가 2번 반복되면 task를 다시 쓴다.
6. `secret`·사내 코드는 실습에 넣지 않는다.
7. 마지막 승인은 사람에게 남긴다.

다음(실습)으로 넘어가며

이제 claude 팀 에이전트 활용 및 `Git worktree` 로 worker의 작업 책상을 나누고, `diff·commit·report·dashboard` 로 결과를 검증한다.

멀티에이전트 오케스트레이션은 “AI를 많이 켜는 기술”이 아니라,
AI 작업 결과를 안전하게 분리·검증·승인하는 업무 운영 방식이다.